Reviewer Pack — Minimal Local Algebraic Geometric Rank and Resolution Proof …

Entry 882661254bfb · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 19:10:14 UTC

Minimal Local Algebraic Geometric Rank and Resolution Proof Size

Entry ID: `882661254bfb` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-31 19:10:14 UTC

1. Conjecture

Field A (mathematical branch): Local Algebraic Geometry **Field B** (complexity object): Resolution Proof Complexity

Statement:

> For every 3-CNF formula φ with n variables, the minimal local algebraic geometric rank (mlag(φ)) of its associated algebraic variety is linearly correlated with its resolution proof size s(φ), such that mlag(φ) = Θ(s(φ)).

Rationale (proposer's reasoning):

> Local algebraic geometry provides a framework to study the complexity of solving polynomial equations over finite fields, which may reveal new insights into the structure of resolution proofs. This conjecture connects the geometric properties of CNF formulas with their proof complexity, potentially leading to new algorithms or lower bounds.

Taxonomy category: `LOCAL_ALGEBRAIC_GEO` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `1da85b666815fb7f`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a set of 30 random 3-CNF formulas, the minimal local algebraic geometric rank (mlag(φ)) and resolution proof size (s(φ)) are linearly correlated such that the Pearson correlation coefficient is ≥ 0.8 for all φ.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"minimal local algebraic geometric rank" AND "resolution proof complexity"` - `"algebraic variety" AND "local algebraic geometry"` - `"3-CNF formula" AND mlag AND s(φ)`

Top relevant hits considered: - [s2:ca44ef9be97a1c211295785ba4b0290db90a6bf2] international workshop on functions in algebra and geometry 2nd-6th 2022 - [s2:10.4171/OWR/2016/57] Asymptotic Phenomena in Local Algebra and Singularity Theory

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def generate_3cnf(n):
    clauses = []
    for _ in range(2 * n):
        literals = [random.choice([1, -1]) * (i + 1) for i in range(n)]
        clause = random.sample(literals, 3)
        clauses.append(clause)
    return clauses

def dpll(clauses):
    def backtrack(level):
        if level == len(clauses):
            return True
        literals = set()
        for clause in clauses[level:]:
            literals.update([abs(lit) for lit in clause])
        literal = random.choice(list(literals))
        sign = 1
        stack.append((literal, sign))
        if backtrack(level + 1):
            return True
        stack.pop()
        sign = -1
        stack.append((literal, sign))
        if backtrack(level + 1):
            return True
        stack.pop()
```

```
        return False

    stack = []
    return backtrack(0)

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = random.choice([5, 10, 15, 20, 30, 40])
    clauses = generate_3cnf(n)
    s_phi = len(dpll(clauses)) if dpll(clauses) else float('inf')

    mlag_phi = Fraction(n)  # Placeholder value for minimal local algebraic geometric rank

    return {
        "metric_name": "mlag_vs_s_phi",
        "metric_value": mlag_phi * s_phi,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    std_value = math.sqrt(sum((res["metric_value"] - mean_value) ** 2 for res in results) / len(re
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(res["seed"] for res in results if not res["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_fai
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6a1bdfd8.py", line 72, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
```

```
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6a1bdfd8.py", line 54, in run_trial
    s_phi = len(dpll(clauses)) if dpll(clauses) else float('inf')
            ^^^^^^^^^^^^^^^^^^
TypeError: object of type 'bool' has no len()
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	23382
2	preregistration	ollama_remote	glm4:latest	0	0	14523
3	novelty	ollama_remote	glm4:latest	0	0	8373
4	novelty	ollama_remote	glm4:latest	0	0	12659
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17216
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14192
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9055
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11979
9	judge	ollama_remote	glm4:latest	0	0	11302

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 122680 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/882661254bfb.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/882661254bfb.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/882661254bfb.tar.gz` (if generated)